SIMATS ENGINEERING

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 3 – CO3

Design a Cloud-Based Collaborative Solution

CO3: Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.

Student detail	Fill in before submission
Student Name	DANIEL SAM K.
Register Number	192524081
Department / Section	B TECH AI & DS
Academic Year / Semester	2ND YEAR
Faculty Name	DR.R.LATHA
Pair member	
Submission Date	5-11-26

Pair Work – 2 Students per Group | Total Marks: 50

Submission note

This document answers all seven questions in the supplied CO3 assignment brief. It follows the brief's terminology and sequence and provides model designs, comparison tables, flow diagrams, API code, security analysis, test plans and a contribution record. Fill in the cover details and adapt the examples to the cloud services actually used by your pair.

Practical evidence must be genuine. This document does not claim that a cloud account was accessed, a live pair-editing session occurred or tests were executed. Add your own original screenshots, terminal outputs, observations and measured results wherever indicated. Do not present proposed or expected results as observed results.

Contents

Section	Topic
Q1	Accessing Cloud Services through Different Methods
Q2	Cloud Platforms and Web Applications
Q3	Web APIs for Cloud Access
Q4	Cloud Security and Network Access
Q5	Collaboration Using Cloud-Based Services
Q6	Integrated Cloud Access and Collaboration Mini Investigation
Q7	Day-4 Design Challenge
Appendix	Contribution record, evidence checklist and references

Q1 – Accessing Cloud Services through Different Methods (6 Marks)

1.1 Introduction

Cloud computing provides computing resources such as storage, applications, databases and processing through a network. A client device sends a request through the Internet or another network to a cloud service. The service authenticates the user, checks permissions, processes the request and returns a response. Cloud services can be accessed through a browser, a mobile application or a program using an API.

1.2 Three cloud access methods

Method	How it works	Uses and evaluation
Browser / cloud console	A user opens a cloud website, signs in and interacts with web pages.	Useful for document storage, administration and web applications. Easy to use and usually cross-platform; requires a compatible browser and network.
Mobile application	An installed phone/tablet app communicates with cloud servers.	Useful for file access, uploads, notifications and approvals. Convenient on the move; depends on app support, device resources and connectivity.
Web API	A program sends HTTP requests to an endpoint and receives structured responses such as JSON.	Useful for automation, integration and bulk operations. Repeatable and scalable, but requires programming, authentication and error handling.

1.3 Architecture and roles



Figure 1. General cloud access architecture

The client accepts user input, displays the interface and sends requests. The network routes requests and responses; bandwidth affects transfer speed and latency affects response time. The cloud service runs application logic, accesses data and enforces identity and permissions. If connectivity is interrupted, cloud access or synchronization may stop until the connection returns.

1.4 Selected service: Google Drive

Google Drive is a suitable example because it supports browser access, mobile access and programmatic integration through supported APIs. A student can upload a document, store it in cloud infrastructure and access it later from another authorized device. Sharing permissions determine whether collaborators can view, comment on or edit the document.

Browser demonstration procedure

- Connect a computer or phone to the Internet.
- Open a supported browser and visit https://drive.google.com.
- Sign in using an authorized account and complete any requested verification.
- Choose New → File upload, select a file and wait for the upload to finish.
- Open Share and invite the collaborator using the correct account and minimum required role.
- Open the file on a second authorized device if possible; sign out on a shared device.

1.5 Requirements and method selection

- Hardware: desktop, laptop, tablet or smartphone with sufficient resources and a working display/input device.
- Software: updated operating system, supported browser or official mobile app, and an authorized user account.
- Network: stable Internet access and permission for secure HTTPS traffic.
- Security: device screen lock, current software, strong account security and MFA where available.

For ordinary student document collaboration, browser access is the most suitable starting method because it offers a full interface without programming. A mobile app is preferable for quick access and notifications. API access is preferable

when software must automate repeated operations or integrate cloud data into another system.

Comparison	Browser	Mobile app	API
Ease of use	High	High	Technical skill required
Automation	Mostly manual	Limited / app-dependent	High
Typical evidence	Login, upload, share screens	App view and file access	Code, request and response
Main limitation	Browser/network dependency	Device/app compatibility	Programming, auth and rate limits

Practical evidence to add: original screenshots of the cloud service, a successful upload, the sharing-permission screen and (if possible) access from a second device. Record actual response time, usability observations and one limitation encountered. Do not invent measurements.

Q2 – Cloud Platforms and Web Applications (7 Marks)

2.1 Selected cloud platforms

Google Cloud Platform (GCP) and Microsoft Azure are selected for comparison. Both offer compute, storage, databases, networking, identity and monitoring services that support web applications. Familiar application examples are Google Drive / Google Workspace and Microsoft OneDrive / Microsoft 365.

Feature	Google Cloud Platform	Microsoft Azure
Compute / hosting	Compute Engine, Cloud Run and App Engine can host application workloads.	Azure Virtual Machines, App Service and Azure Functions can host application workloads.
Storage / data	Cloud Storage and Cloud SQL support files and structured data.	Blob Storage and Azure SQL support files and structured data.
Identity	Cloud IAM and service identities control resource access.	Microsoft Entra ID and Azure role-based access control manage identities and permissions.
Application example	Google Drive / Google Workspace.	OneDrive / Microsoft 365.
Evaluation	Managed services and integration with Google services; plan IAM, quotas and cost.	Broad enterprise and hybrid-cloud capabilities; plan subscription costs, identity and network policies.

2.2 How a cloud platform supports a web application

A cloud platform supplies compute, storage, networking, identity, databases and monitoring. The application contains the user interface and business logic. A user submits a request through a browser; the cloud application validates it, checks identity and permissions, reads or updates cloud data and returns a response. The user generally interacts with the application rather than directly managing the underlying servers.

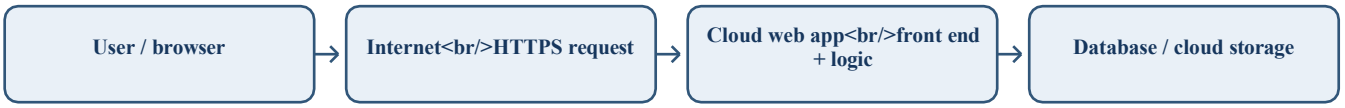


Figure 2. Cloud web-application interaction

2.3 Real-world application: cloud document collaboration

A user signs in, opens a document stored online and edits it. The application synchronizes updates to cloud services. Authorized collaborators can access the same document and, depending on their roles, view, comment or edit it. Version history allows users to inspect earlier document states. This is a cloud web application because the interface, data storage and collaboration features are delivered through an online service.

- Open the application website and authenticate.
- Open an existing document or create a new one.
- Enter or edit content and allow the application to save/synchronize changes.
- Use Share to invite a collaborator and select Viewer, Commenter or Editor as appropriate.
- Ask the collaborator to open the same resource and contribute.
- Use comments or version history to review contributions and previous states.

2.4 Cloud resources involved

- Compute runs application code and processes user requests.
- Storage and databases store files, content and metadata.
- Networking connects the client, application and data services.
- Identity and access management authenticates users and enforces roles.
- Monitoring and logging help track service health, errors and activity.

2.5 Cloud-based versus local-only application

Factor	Cloud-based	Local-only
Access	Multiple authorized devices can access the service over a network.	Often limited to the computer or local network where installed.
Collaboration	Users work on a shared resource subject to permissions.	Often requires exchanging copies or configuring shared storage.
Updates	Provider may deploy updates centrally.	Users or IT staff update each installation.
Data location	Provider-managed cloud infrastructure.	Local disk or organization-managed server.
Availability	Depends on network and provider availability.	Can work offline if application and data are local.
Control	Depends on provider and customer configuration.	More direct local control, but maintenance is the user's responsibility.

2.6 Evaluation

Cloud applications improve accessibility, centralized maintenance, collaboration and scalability. Their limitations include dependence on network connectivity, provider availability, subscription cost and correct permission settings. A local application may be preferable when offline operation, direct control of infrastructure or specialized hardware integration is essential. The choice should be based on user needs, data sensitivity, connectivity, cost and operational requirements.

Evidence to add: screenshots or demonstrations of the selected services, a user-action sequence and observations comparing cloud use with a local-only approach.

Q3 – Web APIs for Cloud Access (7 Marks)

3.1 Meaning and purpose of a Web API

A Web API is a defined interface that allows software components to communicate over a network. A client sends an HTTP request to an endpoint using a method such as GET or POST. The server processes the request and returns a response containing a status code, headers and often a JSON body. APIs let applications access cloud services programmatically without a person manually operating the website.

3.2 Selected public test API: JSONPlaceholder

JSONPlaceholder (https://jsonplaceholder.typicode.com) is a public test API that provides sample resources such as posts, users and comments. It is suitable for learning HTTP requests and JSON parsing. It is a demonstration API, not a production database for private student information. This example retrieves two sample posts using GET.

Request component	Description	Example
Endpoint	Identifies the requested resource.	https://jsonplaceholder.typicode.com/posts/1
HTTP method	Specifies the operation.	GET
Headers	Metadata such as accepted response type or authorization.	Accept: application/json
Status code	Indicates request outcome.	200 = successful response
Response body	Data returned by the API.	JSON containing id, title, body and userId

3.3 API communication flow

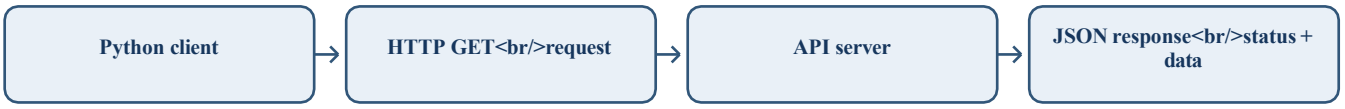


Figure 3. API request and response flow

The client constructs a URL and sends the request. The API server validates it, locates the resource and returns a response. The client checks the status code, parses JSON and displays selected fields. A robust client handles timeouts, unavailable services and invalid responses instead of crashing.

3.4 Python implementation

Install the Requests library with **python -m pip install requests**. Then run the following program. It makes two requests, one for post 1 and one for post 2.

```
import requests

BASE_URL = "https://jsonplaceholder.typicode.com/posts/"

for post_id in (1, 2):
    url = BASE_URL + str(post_id)
    try:
        response = requests.get(url, timeout=10)
        print("Request URL:", url)
        print("HTTP status:", response.status_code)
        response.raise_for_status()
        post = response.json()
        print("Post ID:", post["id"])
        print("Title:", post["title"])
        print("User ID:", post["userId"])
        print("-" * 40)
    except requests.exceptions.RequestException as error:
        print("Request failed:", error)
    except (ValueError, KeyError) as error:
        print("Invalid response:", error)
```

Test	Request	Expected response
1	GET /posts/1	Normally HTTP 200 and JSON object with id 1.
2	GET /posts/2	Normally HTTP 200 and JSON object with id 2.

The exact title and response body must be copied from your own program output after execution. The public service may be unavailable or change; therefore, do not present assumed output as a measured result.

3.5 Why API access is useful for many users

APIs provide a consistent interface for browsers, mobile apps, partner systems and background services. They enable automation and integration, centralize business logic and avoid requiring every client to implement the same service independently. Cloud platforms can scale application instances and supporting resources as demand grows, subject to service capacity, rate limits and cost. Good API design includes authentication, authorization, input validation, monitoring, versioning and rate limiting.

3.6 Analysis and limitations

- Advantages: automation, repeatability, interoperability, centralized logic and integration with other systems.
- Limitations: network dependency, rate limits, API version changes, authentication requirements and service outages.
- Good practice: use HTTPS, timeouts, status-code checks, response validation and safe secret storage; never hard-code private tokens in public code.

Evidence to add: the Python source file, terminal screenshots showing both actual requests and outputs, and a short analysis of the responses.

Q4 – Cloud Security and Network Access (7 Marks)

4.1 Introduction

Cloud services are reachable over networks, so identity verification, permission management and secure communication are essential. Security follows a shared-responsibility model: the provider protects the underlying service infrastructure, while the customer must configure accounts, permissions, devices and data handling correctly.

4.2 Security considerations

Consideration	Meaning	Suitable control
Authentication	Verifies the identity of a user or service.	Strong passwords, MFA, single sign-on and secure identity provider.
Authorization	Determines what an authenticated identity may access or change.	Role-based access control, least privilege and regular permission review.
Secure communication	Protects data while travelling between client and cloud.	HTTPS/TLS, valid certificates and avoiding plain HTTP.
Data protection	Protects stored and transferred information.	Encryption at rest, backups, retention rules and restricted sharing.
Monitoring and audit	Helps detect suspicious access and investigate events.	Activity logs, alerts, access reviews and incident response.
Device security	A compromised device can expose cloud sessions and data.	Screen lock, software updates, endpoint protection and safe sign-out.

4.3 Authentication versus authorization

Authentication answers “Who are you?” A user proves identity using an account credential and, where enabled, a second factor. Authorization answers “What are you allowed to do?” After authentication, the service checks the user's role and resource permissions. A user may be authenticated successfully but still be denied access because the account lacks permission to view or modify a particular resource.

4.4 Secure access flow



Figure 4. Secure cloud access sequence

4.5 Threats and mitigations

Threat	Impact	Mitigation
Stolen password / phishing	Attacker may impersonate a user.	MFA, phishing awareness, sign-in alerts and strong authentication.
Over-permissive sharing	Unauthorized people may view, edit or download files.	Least privilege, restricted links and periodic access review.
Insecure network traffic	Data may be intercepted or modified.	HTTPS/TLS, certificate validation and trusted networks.
Compromised device	Session tokens or downloaded files may be stolen.	Patch devices, screen lock, endpoint protection and revoke sessions.
Exposed API keys	Attacker may call cloud services using leaked credentials.	Secret manager, scoped permissions, rotation and monitoring.
Denial of service	Legitimate users may be unable to access service.	Rate limits, monitoring, provider protections and continuity planning.

4.6 Secure versus insecure access

Secure access uses HTTPS, individual accounts, MFA and role-based permissions. An insecure situation might use a shared password, an unrestricted public link or plain HTTP. Insecure practices make it difficult to identify who performed an action and can expose information to unauthorized users. Consequences include data leakage, accidental deletion,

service interruption, reputational damage and institutional or legal consequences.

4.7 Why cloud access needs additional measures

A local application may be limited to a particular computer or internal network. A cloud application can be reached from many locations and devices, increasing convenience and the possible attack surface. Cloud users therefore need strong identity verification, least-privilege authorization, secure communication, device protection, monitoring and recovery procedures.

Evidence to add: document the security settings used by your platform, show test-user roles or sharing permissions, and compare an authorized user's access with a user who lacks permission. Hide private account information and never expose passwords or tokens.

Q5 – Collaboration Using Cloud-Based Services (6 Marks)

5.1 Selected service and requirements

Google Docs with Google Drive is selected as the example collaboration service. The pair creates one shared document in a cloud folder. Both students access the same online resource instead of exchanging separate copies. Requirements include shared access, clear user roles, concurrent editing, version history and a record of contributions.

5.2 Collaboration workflow

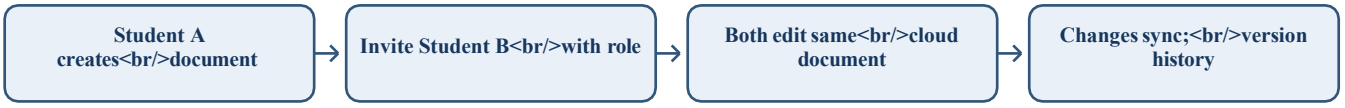


Figure 5. Shared-document workflow

5.3 Sharing permissions

Role	Permission	Recommended use
Owner	Controls the file and sharing settings.	One designated group member or institutional account.
Editor	Can modify document content.	Pair members who must jointly edit.
Commenter	Can add comments or suggestions but not directly edit main content.	Reviewer or faculty guide.
Viewer	Can read but cannot edit.	Users who only need to inspect the final work.

Use restricted sharing and invite only the intended accounts. Give Editor access only to people who need to modify the file. Avoid public link sharing unless specifically required. Review the access list before submission.

5.4 Two collaborative activities

- Joint editing: Student A creates the document structure and writes the problem statement. Student B opens the same document and adds the platform comparison. Both review the shared version and resolve overlapping edits.
- Comment and review: Student B adds a comment asking for clarification of the API section. Student A responds and updates the paragraph. The pair reviews version history to confirm that the changes were saved.

5.5 Concurrent access and version control

Concurrent access allows multiple authorized users to work on the same resource. The application synchronizes changes and may show collaborator presence or editing indicators. Version history stores earlier states or activity records so users can inspect or restore a previous version when supported. Users should communicate before large structural changes to avoid confusion.

5.6 Benefits and common issues

Area	Explanation / response
Fewer duplicate copies	Both users edit one shared resource, reducing conflicting attachments and outdated versions.
Real-time visibility	Changes become visible to collaborators after synchronization.
Incorrect permissions	An editor may change or delete content; a public link may expose private information. Use least privilege.
Conflicting edits	Communicate, use comments/suggestions and review version history.
Network interruption	Changes may not synchronize immediately. Check connectivity and confirm saving before closing.

5.7 Evidence to add

Include screenshots of the shared document, the sharing/permission dialog, two visible contributions and version history or comments. Record which student completed each activity. The contribution record must reflect work actually performed.

Q6 – Integrated Cloud Access and Collaboration Mini Investigation (7 Marks)

6.1 Scenario: Cloud-Based Student Project Management

Problem: Student project teams often coordinate tasks using separate messages, local files and spreadsheets. This can cause duplicate files, missed deadlines and unclear task ownership. The proposed system provides one cloud workspace where students and faculty can view project information, update task status and collaborate on shared documents.

6.2 Users and requirements

User	Responsibilities
Student	View assigned tasks, update progress, upload files and collaborate.
Team leader	Create tasks, assign members, set due dates and review progress.
Faculty / guide	Review milestones, provide comments and monitor project status.
Administrator	Manage accounts, access roles, configuration and backups.

Functional requirements

- Users sign in and see only projects they are authorized to access.
- Team leaders create projects, assign tasks and set due dates.
- Students update task status and upload deliverables to cloud storage.
- Team members edit shared documents and post comments.
- Faculty review progress and provide feedback.

Non-functional requirements

- Availability: accessible when needed, subject to network and provider availability.
- Security: authentication, role-based authorization and protected communication.
- Scalability: support additional teams and files as usage grows.
- Usability: simple interface for students and faculty.
- Reliability: backups, logging and recovery procedures.

6.3 Proposed architecture

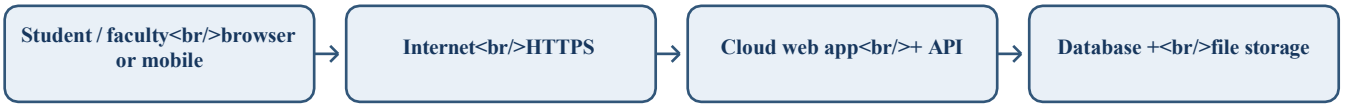


Figure 6. Student project management architecture

The client communicates with the cloud application over HTTPS. The application exposes web pages and API endpoints. The API validates input and checks user roles before updating project records in a managed database or storing files in cloud object storage. A collaboration service supports shared editing. Monitoring and backups support operations and recovery.

6.4 Required cloud services

Service	Purpose
Application hosting	Runs the web application and API, for example on a managed app or container platform.
Cloud database	Stores users, projects, tasks, assignments, deadlines and status.
Object storage	Stores reports, presentations and other uploaded project files.
Identity and access	Authenticates users and assigns student, faculty and administrator roles.

Service	Purpose
Collaboration / notifications	Supports shared documents, comments and task notifications.
Monitoring and backup	Tracks service health and supports recovery.

6.5 User workflow

- A student opens the application using a browser or mobile device and signs in.
- The application authenticates the user and retrieves only authorized projects.
- The team leader creates a task, assigns it and sets a due date.
- The student uploads a draft to cloud storage and changes the task status.
- A team member edits the shared document or adds comments.
- The faculty guide reviews the deliverable and provides feedback.

6.6 Prototype / mock-up screens

Screen	Proposed elements
Login	Institutional email, password or SSO, and MFA where enabled.
Dashboard	Project name, team members, task counts, due dates and progress.
Task details	Title, description, assignee, due date, status and attachment link.
Shared files	File list, upload control, sharing status and version information.
Faculty review	Submission preview, comment field, review status and feedback history.

6.7 Two user test scenarios

Test	Steps	Expected behaviour	Actual result
Student submission	Student signs in, opens assigned task, uploads a file and marks it Submitted.	Authorized student can access task, upload file and see updated status.	Complete after actual test.
Faculty review	Faculty signs in, opens project, views submission and adds feedback.	Faculty can review and save feedback; student can later view it.	Complete after actual test.

6.8 Evaluation and improvements

Combining cloud access and collaboration improves efficiency because users reach the same project workspace from authorized devices and work on shared resources without repeatedly exchanging copies. Task status and feedback are centralized, improving visibility and accountability. Limitations include Internet dependency, service outages, subscription costs, file-size limits and permission errors. Improvements could include offline draft support, automated backups, audit logs, notification controls, accessibility improvements and integration with institutional identity systems.

Evidence to add: prototype screenshots, one cloud-access demonstration, one collaboration demonstration and actual results for both user scenarios. State clearly which features are only proposed and which were implemented.

Q7 – Day-4 Design Challenge: Cloud-Based Collaborative Solution (10 Marks)

7.1 Problem and target users

The final joint design is a Cloud-Based Student Project Management and Collaboration System. It addresses the difficulty of coordinating tasks, sharing deliverables and collecting faculty feedback when students use disconnected tools and multiple file copies. Target users are students, team leaders, faculty guides and system administrators.

7.2 Stage 1 – Independent proposals

The assignment requires Student A and Student B to prepare independent proposals before discussion. The following model outlines are examples only. The pair should retain their own original notes or dated drafts as evidence and adapt the proposals to reflect the ideas they actually developed independently.

Design element	Student A – example	Student B – example
Problem	Task allocation and progress tracking are scattered across messages and local files.	Project documents and faculty feedback are difficult to track in one place.
Users	Students, team leaders and faculty guides.	Students, faculty reviewers and administrators.
Access	Browser application with optional mobile access.	Mobile-friendly web application and API integrations.
Cloud services	Managed hosting, database, object storage and identity.	Cloud hosting, shared document service, notifications and audit logs.
Collaboration	Shared task board, comments and project folder.	Collaborative documents, review comments and version history.
Architecture	Browser → HTTPS → cloud app/API → database and file storage.	Mobile/browser → secure network → cloud platform → collaboration services.

7.3 Stage 2 – Design comparison record

Area	Comparison	Joint decision
Agreement	Both use cloud hosting, browser/mobile access and shared resources.	Retain a common cloud workspace and role-based access.
Difference	A emphasizes task tracking; B emphasizes documents and feedback.	Combine task management with collaborative document review.
Strengths	Central task ownership and deadlines; shared editing and feedback.	Use task records for accountability and shared documents for content.
Security	Unauthorized sharing, weak passwords and excessive roles.	Use MFA, HTTPS, least privilege and restricted sharing.
Missing items	Backup, monitoring, audit trail and outage handling.	Add logs, backups, monitoring and recovery procedures.

7.4 Stage 3 – Joint final design

The pair combines the strongest elements into one system. It uses a responsive web client, HTTPS communication, cloud-hosted application/API services, a managed database, object storage and a collaboration service. Users authenticate before access, and the application checks each user's role before allowing project, task or file operations.



Figure 7. Joint final architecture

Complete system workflow

- The user opens the web or mobile interface and authenticates using an institutional identity provider or approved account mechanism.
- The application checks the user's role and project membership before returning data.
- A team leader creates a project, defines tasks, assigns owners and sets deadlines.

- Students upload files to object storage; the database stores project and file metadata.
- Authorized team members edit shared documents, add comments and review version history.
- Faculty view submissions, provide feedback and update review status.
- Relevant actions are logged and data is backed up according to the institution's retention policy.

7.5 Web application and API requirements

The web application provides login, dashboard, task management, file upload and faculty review screens. Example API operations include GET /projects, GET /projects/{id}/tasks, POST /tasks, PATCH /tasks/{id}, POST /files and POST /feedback. Each operation must validate input, authenticate the caller, authorize the action, validate uploaded content and return an appropriate HTTP status. Secrets must not be embedded in public client-side code.

7.6 Authentication, security and data sharing

- Require individual accounts and MFA where available.
- Use HTTPS/TLS for all client-to-cloud communication.
- Apply role-based access: students access their team projects, faculty access assigned projects and administrators manage configuration.
- Use least-privilege file permissions and avoid unrestricted public links.
- Validate input and uploaded file types; apply file-size limits and malware scanning where supported.
- Store secrets securely, rotate credentials and maintain audit logs.
- Back up project metadata and files and test restoration procedures.

7.7 Stage 4 – Validation plan

Test	Actor / condition	Expected result	Evidence
TC1 Authorized access	Student who belongs to project.	Dashboard and permitted tasks appear.	Dashboard screenshot.
TC2 Unauthorized access	User without project membership.	Access denied; project data not disclosed.	Denied response or log.
TC3 File upload	Authorized student uploads allowed file.	File stored and listed in project.	Upload confirmation.
TC4 Collaboration	Two authorized users edit/comment.	Changes synchronize and contributions can be reviewed.	Comments/version history.
TC5 Faculty feedback	Assigned faculty reviews submission.	Feedback saved and visible to student.	Review screen.

Student A – technical validation

Student A explains why browser-based access was selected, how HTTPS connects the client to the cloud application, how the API communicates with database and storage services, and why the chosen services satisfy the requirements. Student A should identify platform trade-offs, service dependencies and cost limitations.

Student B – experimental validation

Student B prepares and executes test cases using test accounts, records expected and observed behaviour, captures screenshots and documents failures. Actual unsuccessful tests must be reported honestly rather than changed to match expected results.

7.8 Stage 5 – Cross-defence

Question	Model answer
What experimental evidence supports the design?	Actual evidence should include authorized sign-in, denial of access to an unauthorized user, file upload confirmation, shared editing/comment record and faculty feedback record. The pair must show screenshots or logs collected during testing.
Why was this access and collaboration approach selected?	A responsive browser application works across common devices without a separate installation. HTTPS protects communication, cloud storage centralizes files, shared documents support collaboration and role-based access protects resources.

7.9 Stage 6 – Joint evaluation

Factor	Assessment
Benefits	Centralized project information, multi-device access, fewer duplicate files, clear task ownership and faster feedback.
Access limitations	Internet dependency, latency, provider outages, device/browser compatibility and account availability.
Collaboration	Shared editing, comments, version history and visibility of task progress.
Security risks	Account compromise, incorrect permissions, exposed API credentials, malware uploads and data leakage.
Scalability	Managed hosting and storage can expand as users and projects grow, subject to quotas, design and budget.
Improvements	Offline drafts, analytics, calendar integration, reminders, accessibility testing and disaster-recovery exercises.
Not appropriate when	Offline operation is mandatory, cloud transfer is prohibited, connectivity/cost is unacceptable or requirements cannot be met.

7.10 Final answer – characteristics a designer must consider

A system designer should evaluate user types and numbers, devices, network availability, usability, data sensitivity, authentication, authorization roles, collaboration features, storage needs, API integration, scalability, availability, backup and recovery, compliance, vendor dependence and total cost. The designer must also consider offline access, synchronization, permission reviews and incident detection. Cloud access and collaboration are suitable when their benefits outweigh network, security, cost and provider-dependency risks.

Appendix A – Student Contribution Record

Complete this table using the actual work performed by both students. Primary contribution does not mean exclusive responsibility; both students must understand the complete assignment.

Question	Student A – primary contribution	Student B – primary contribution	Joint work
Q1	Access-method design and architecture	Practical access demonstration and observations	Service selection and comparison
Q2	Platform analysis and interaction diagram	Application investigation and demonstration	Local-versus-cloud evaluation
Q3	API purpose, flow and request/response design	Python implementation and execution evidence	API access analysis
Q4	Security investigation and comparison	Security design, threats and mitigations	Secure-access evaluation
Q5	Collaboration workflow and permissions	Collaborative activities and contribution evidence	Review of sharing and version history
Q6	Problem definition, users and architecture	Prototype, user tests and evaluation	Mini-investigation conclusions
Q7	Technical validation and platform justification	Experimental validation and test evidence	Independent proposals, joint design and defence

Appendix B – Practical evidence checklist

- Q1: browser/mobile access, upload/share screenshots and observations.
- Q2: two-platform comparison and user-action sequence.
- Q3: Python code, two actual request outputs and execution screenshot.
- Q4: authentication and permission evidence plus threat/mitigation analysis.
- Q5: shared workspace, role settings, two collaborative activities and contribution evidence.
- Q6: prototype, cloud access and collaboration demonstrations, and two actual user-test results.
- Q7: independent proposals, comparison record, final architecture, test results and cross-defence preparation.

Appendix C – Viva preparation

Possible question	Key point
What if Internet connectivity is unreliable?	Cloud access may slow or fail; retry logic, local drafts or offline support may help where available.
What if everyone has Editor permission?	Users may accidentally or deliberately change/delete content; apply least privilege and review access.
What if the API is unavailable?	Handle timeouts/errors, show a useful message, retry safely and protect unsaved input.
What if a user is authenticated but lacks permission?	Authentication succeeds but authorization fails; deny the requested action.
Why use an API instead of manually opening a website?	APIs enable software to perform repeatable operations and integrate services automatically.

References and learning resources

Use current official documentation to verify service features and implementation details because names, interfaces and limits may change.

- Google Drive Help – https://support.google.com/drive/
- Google Cloud Documentation – https://cloud.google.com/docs
- Microsoft Azure Documentation – https://learn.microsoft.com/azure/
- Microsoft 365 Support – https://support.microsoft.com/microsoft-365
- Python Requests Documentation – https://requests.readthedocs.io/
- JSONPlaceholder – https://jsonplaceholder.typicode.com/
- MDN Web Docs, HTTP overview – https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
- OWASP security resources – https://owasp.org/

Conclusion

Cloud-based solutions combine network access, hosted applications, cloud storage, web APIs and collaboration services to support real-world work. These seven tasks explain browser, mobile and API access; cloud platforms and web applications; programmatic communication; security controls; shared workspaces; and integrated collaborative system design. The proposed student project management system brings these concepts together. Its success must be demonstrated through genuine implementation evidence, actual test outcomes and clear records of both students' contributions.